Client Audit Implementation Plan

This plan turns `CLIENT_AUDIT_REPORT.md` into a staged rollout so urgent risk reduction happens first, while broader refactors stay reviewable and reversible.

Guiding Principles

1. **Fix confirmed security issues before maintainability work.**
2. **Prefer small, testable batches over one large refactor.**
3. **Treat client/server boundary issues as one change set when they must evolve together.**
4. **Validate audit findings before changing code** where the report may already be partially outdated.

Phase 1 — Security & Correctness Stabilization

Goal: Remove the highest-risk confirmed issues without reshaping the app.

Scope

- Sanitize CMS HTML before rendering in `StaticPageView`.
- Replace email verification API usage from query-string transport to request-body transport.
- Move newly generated verification links to URL fragments so tokens are not sent to the server in the initial browser request.
- Scrub verification tokens from the browser URL after capture.
- Verify whether messaging credentials still round-trip in plaintext; only change behavior if confirmed by current code/API behavior.

Why first

- These issues have the highest blast radius.
- They are relatively isolated.
- They reduce risk before we begin larger refactors.

Acceptance criteria

- Static CMS content is sanitized before render.
- Future verification emails use `#token=...`, not `?token=...`.
- Client verifies via `POST /auth/verify-email` with `{ token }`.
- Verification page clears token material from the visible URL after reading it.
- Legacy query-string links still work during transition.

Phase 2 — Dead Code, Duplication, and Low-Risk Cleanup

Goal: Reduce noise and remove easy sources of drift before deeper redesign.

Scope

- Delete dead `AllOrdersPage/AllOrdersPage.jsx`.
- Remove stray comments and empty/stub files where safe.
- Extract duplicated utilities:
  - `generateSlug`
  - `toDateTimeLocal`
  - `getTaxRows`
  - `formatDateOnly`
  - `isExternalUrl`
- Extract `useScrollState` from repeated horizontal-scroll logic.
- Add `React.memo` to genuinely pure shared components.
- Centralize simple constants such as store name and locale defaults.

Why second

- These are low-risk improvements that make later work easier to review.
- They immediately reduce duplicate maintenance surfaces.

Acceptance criteria

- Shared helpers live in one place and callers use them.
- Dead code is removed rather than preserved “just in case.”
- No behavior change unless intentionally documented.

Phase 3 — Service Layer and Error Handling Normalization

Goal: Make data access predictable across the client.

Scope

- Move direct `api.*` calls out of admin pages into service modules.
- Standardize service export style and response shape handling.
- Centralize URLSearchParams construction.
- Introduce a consistent user-facing error pattern instead of scattered `console.error`.
- Add request cancellation where pages can race during rapid navigation/filter changes.
- Decide whether lightweight retry logic belongs in the axios layer.

Why here

- Once duplication is lower, service boundaries become easier to standardize.
- This phase improves maintainability without yet touching the largest components.

Acceptance criteria

- Admin pages no longer call raw `api.*` directly except where explicitly justified.
- Services follow one documented convention.
- Error handling is consistent across representative pages.

Phase 4 — Performance, UX, and Accessibility Pass

Goal: Improve user experience and runtime behavior without changing business logic.

Scope

- Memoize `Intl.NumberFormat` in `useCurrency`.
- Replace brittle effect dependencies and obvious render churn hotspots.
- Improve loading and empty states on auth/admin pages.
- Fix touch targets, `aria-label`s, contrast issues, and `prefers-reduced-motion`.
- Replace native `alert` / `prompt` interactions with app-native UI.
- Revisit data caching strategy for shared contexts.

Why after service normalization

- Performance and UX work is easier when data flows are already more predictable.

Acceptance criteria

- Known audit regressions are addressed with visible UX improvements.
- Accessibility issues are reduced in the highest-traffic components.
- Avoid introducing a caching library until the team agrees on the pattern.

Status

- **Completed:** memoized `useCurrency`, auth/admin loading improvements, touch-target/contrast/reduced-motion fixes, app-native dialog replacements, `MediaPicker` race/cache cleanup, dynamic store-name handling, date-format consolidation, and `BrandStrip` dark-mode cleanup.
- **Deferred to Phase 5:** the larger render-churn findings tied to oversized page state (`ProductEditPage`, `CheckoutPage`) and broad page-level decomposition work.
- **Closeout note:** the remaining high-effort client structure issues are no longer small Phase 4 fixes; they are better handled with tests in place during Phase 5 decomposition.

Phase 5 — Large Component Decomposition

Goal: Break up the files that are currently expensive to reason about and risky to change.

Scope

- `ProductEditPage`
- `SettingsPage`
- `OrderDetailPage`
- `ApiBuilderPage`
- `CheckoutPage`
- `AdminLayout`
- `StoreLayout`

Strategy

- Extract by responsibility, not by arbitrary line count:
  - form sections
  - derived state/hooks
  - table/config helpers
  - layout subtrees
- Add focused tests around extracted behavior before or during decomposition.

Why last

- This is the highest-effort phase and benefits from all earlier cleanup.
- Decomposing large files before normalizing shared utilities usually multiplies inconsistency.

Acceptance criteria

- Each extracted unit has a clear purpose.
- Parent pages become orchestration layers rather than dumping grounds.
- High-risk flows remain behaviorally equivalent.

Phase 6 — Test Foundation

Goal: Add enough client-side automated coverage to make future refactors safer.

Scope

- Add Vitest + React Testing Library setup.
- Start with tests for:
  - auth flows

  - shared utilities
  - key hooks
  - security-sensitive rendering paths
- Add a few smoke tests around representative pages/components rather than attempting full coverage at once.

Why not first

- The project currently has no client test harness, and the most urgent issues are already identifiable without one.
- Adding tests becomes more valuable once shared utilities and service conventions settle.

Acceptance criteria

- `npm test` (or equivalent) exists for the client.
- Core helpers and security-sensitive behavior have regression coverage.

Initial starter batch

- Add Vitest + React Testing Library setup.
- Add fast regression tests for shared utilities introduced during cleanup.
- Add one security-sensitive rendering test for sanitized CMS content.

Recommended Execution Order

1. **Phase 1** — immediate risk reduction
2. **Phase 2** — quick cleanup wins
3. **Phase 3** — service consistency
4. **Phase 4** — performance and UX
5. **Phase 6** — test harness before the largest refactors
6. **Phase 5** — major decomposition with tests in place

> Note: Phase 6 is listed after Phase 5 above for completeness, but in practice it should begin before or alongside the biggest Phase 5 refactors.

First Working Batch

The first implementation batch should include:

1. `StaticPageView` sanitization
2. Safer email verification flow
3. A short verification note on messaging credential handling so we decide whether that needs code or only documentation

That gives us a meaningful security improvement without pulling the whole codebase into one risky branch.